

XORZEN Forensic Audit Report

Independent Architecture Verification and Correctness Assessment

Repository: github.com/akikfaraji/DevNet

Commit: 4b2bc57c806059b49b3ad375293147b62e133575

Branch: main

Date: 2026-09-01

Audit Type: Skeptical senior ML systems research audit

This report establishes, from the actual current repository state, what XORZEN really is, what works, what does not work, what is measurable, and which claims are supported by evidence. The goal is truth, not validation.

A. Executive Summary

XORZEN (zeroModel) is a conditional-compute transformer architecture that combines three parallel pathways (Local Attention, Low-Rank Global Attention, and SSM), a Mixture-of-Experts (MoE) layer, an adaptive router controlling depth/width/pathway/expert routing, and an internal Chain-of-Thought (CoT) reasoning module. The architecture is ambitious in scope, incorporating adaptive depth, adaptive width, pathway selection, expert routing, cost-aware budgeting, and latent reasoning vectors into a single model. The codebase consists of approximately 140 files across model definitions, training infrastructure, tokenization, utilities, benchmarking scripts, and extensive historical audit reports documenting iterative debugging over multiple versions.

The model does not currently produce correct forward passes. A critical shape bug in LowRankGlobalPathway (hass_block.py line 296) causes causal_mask broadcasting to introduce a spurious 4th dimension in the output tensor. This bug propagates through the entire forward path, causing SlicedFFN and downstream components to crash with shape mismatches. The existing test suite confirms this: 40 of 138 tests (29%) fail, including all causal correctness tests, all CoT tests, all sparse dispatch tests, and multiple routing/gradient tests. The model cannot be benchmarked, trained, or evaluated in its current state without first fixing this blocking bug. This means that all performance claims in existing reports are based on code that no longer functions at HEAD.

Beyond the blocking forward-pass bug, the audit identified 6 critical issues, 12 high-severity issues, and numerous medium/low concerns. The most serious include: dead parameters that inflate parameter counts (context_weights, injection_gate), a cost-aware routing bias inversion that biases toward expensive pathways under low budget, incorrect active-parameter accounting in forward_with_depth that routes all tokens to expert 0, a SlicedFFN inconsistency where ln_hidden is applied in one code path but not another, and 6 instances of torch.load without weights_only=True creating arbitrary code execution vulnerabilities. No claims about inference speedup, training efficiency, or quality improvement can be verified because the model cannot execute a forward pass.

Overall maturity: Experimental prototype. The architecture contains genuinely interesting research ideas (conditional compute across multiple axes, internal latent reasoning, cost-aware routing), but the implementation has not reached a state where any scientific claims can be evaluated. The most promising research direction is the multi-axis adaptive routing system, which, if stabilized and made correct, could enable meaningful compute savings. The first priority must be fixing the blocking shape bug and achieving a clean forward/backward pass before any further architectural development or benchmarking is attempted.

B. Repository Snapshot

B.1 Commit and Branch

Commit: 4b2bc57c806059b49b3ad375293147b62e133575

Branch: main (only branch)

Commit message: "fix: LowRankGlobalPathway causal leakage + pre-training correctness audit"

No tags or releases exist. Working tree is clean.

B.2 Compute Environment

Property	Value
OS	Debian GNU/Linux 13 (trixie)
Kernel	5.10.134-013.8.3.kangaroo.al8.x86_64

CPU	Intel Xeon, 2 physical cores, 2 logical
RAM	4.1 GiB (3.6 GiB available)
GPU	NONE
Python	3.12.14
PyTorch	2.13.0+cpu
CUDA	Not available (CPU-only)
NumPy	2.5.2
transformers	5.15.1
tokenizers	0.22.2
einops	0.8.2

Critical limitation: No GPU available. All benchmarks are CPU-only, toy-scale. No realistic-scale inference or training benchmarks are possible on this hardware. All benchmark results are labeled as toy-scale and cannot be extrapolated to GPU deployment scenarios.

B.3 Repository Structure

The repository contains 140+ files organized into: `xorzen/model/` (core architecture), `xorzen/models/zero/` (zeroModel variant), `xorzen/models/igris/` (experimental IGRIS variant), `xorzen/training/` (training loop), `xorzen/tokenizer/` (BPE tokenizer), `xorzen/data/` (data pipeline), `xorzen/utils/` (SPPQ, sharding, math), `xorzen/speed/` (C++ acceleration layer), `tests/` (138 tests), `scripts/` (audit and experiment scripts), `benchmarks/` (benchmark utilities), `reports/` (29+ historical JSON/MD reports), and `experiments/` (training evidence). The codebase has extensive self-auditing infrastructure but the actual model is broken at HEAD.

C. Architecture Reconstruction

The following describes the actual implementation-level architecture as traced from the code at commit 4b2bc57. This is not what the README or documentation claims; this is what the code actually does.

C.1 Forward Pass Execution Path

Input: `input_ids [B, T]` of integer token IDs. Embedding: `token_embedding + position_embedding` (learned, not RoPE) with dropout. CoT: `InternalLatentCoT` is initialized but DISABLED during pre-training (frozen, outputs zeros). The 6 CoT components (intention, decomposition, confidence, contradiction, direction, summary) each project through `Linear(H, cot_dim*2)` with GLU gating, component-specific activations, and a GRU updater. The `injection_gate` parameter is defined but never called in `forward()` -- it is dead code. The CoT output is zeroed during pre-training, so it contributes nothing to the forward pass but its parameters are still counted.

Routing: `AdaptiveRouter` takes `[B, T, H+cot_dim]` and produces: `depth_mask [B, T, max_depth]` via Gumbel-sigmoid STE, `width_probs/idx/multiplier [B, T, num_widths]` via Gumbel-softmax, `path_probs [B, T, 3]` via Gumbel-softmax with exploration temperature, `expert_indices/weights [B, T, top_k]` via top-k selection with capacity constraint. The router has a 3-layer MLP feature encoder, 5 routing heads (depth, width, path, expert, complexity), and an uncertainty estimator. Cost-aware routing biases decisions based on a `compute_budget` scalar, but the pathway bias is inverted (see weaknesses).

HASS Blocks: Each of `num_layers` blocks contains 3 parallel pathways and a `SlicedFFN`. The block receives `routing_decision` and, when `pathway_top_k >= 3` (default), computes all 3 pathways and blends with `path_probs`. When `pathway_top_k < 3`, it uses `sparse_pathway_dispatch` to execute only top-k pathways. Depth masking at inference uses gather-scatter to skip inactive tokens per layer. The 3

pathways are: (1) LocalAttentionPathway: windowed multi-head attention with half the model heads, causal + window masking, per-head Q/K LayerNorm. (2) LowRankGlobalPathway: projects to low-rank space, performs full or chunked causal attention, projects back. Contains a dead parameter (context_weights). (3) SSMPathway: Mamba-style diagonal-A SSM with ZOH discretization, input-dependent timestep, center-padded Conv1d (documented causal violation of +/-1 position), gated output.

SlicedFFN: Genuine per-token width sparsity via matrix slicing. `fc1.weight[:w, :]` and `fc2.weight[:, :w]` are used to compute smaller matmuls for tokens selecting width `w`. Width choices are derived from `config.width_choices` plus the `max_width` (`hidden_size * expert_hidden_multiplier`). Inconsistency: `ln_hidden` is applied at `max_width` in `_forward_single_width` but never in `_forward_single_width_flat` (the per-token path).

MoE: ShardedExpertFabric with disk-backed LRU-cached experts. In `test_mode`, uses a single dummy ExpertFFN (SwiGLU: `gate_proj`, `up_proj`, `down_proj`). In production mode, experts are loaded from disk on demand with an LRU cache. Top-k tokens per slot are grouped by expert and processed. Load balance loss uses an incorrect L2 formula (not Switch Transformer), separate from the canonical Switch formula in the router.

Merger: GatedMerger concatenates [HASS; MoE; CoT], passes through a 2-layer MLP gate controller producing 3 gate logits, softmax normalizes, and computes weighted sum. The `attention_mask` parameter is accepted but never used. Output: `final_norm` (LayerNorm) then `lm_head` (Linear, weight-tied with `token_embedding`) producing logits [B, T, V]. Loss: CrossEntropyLoss with `ignore_index=pad_token_id`, shifted for next-token prediction. Auxiliary losses: `routing_loss` (uncertainty regularizer), router auxiliary losses (`load_balance_loss` via Switch formula, `router_z_loss`, `path_div_loss`, `width_div_loss`), optionally CoT consistency loss (disabled during pre-training).

D. Execution Graph

The execution graph could not be fully traced at runtime because the forward pass is blocked by the LowRankGlobalPathway shape bug. The following is reconstructed from code analysis. At inference with default settings (`pathway_top_k=3`, `test_mode=True`, CoT disabled), the execution path for a single layer is: (1) LayerNorm on input, (2) All 3 pathways computed in parallel (Local + LowRank + SSM), (3) Weighted sum with `path_probs`, (4) Residual addition, (5) LayerNorm, (6) SlicedFFN with per-token width from router, (7) Residual addition. For depth masking, if some tokens have `depth_mask=0` at a given layer, the model uses `forward_with_depth` which gathers active tokens, processes them, and scatters back. However, `forward_with_depth` creates a mini routing decision with all `expert_indices` set to 0, meaning all depth-masked tokens lose their per-token expert routing and go to expert 0. This is a correctness bug in the depth-sparse path, but it is currently moot because the forward pass fails before reaching depth masking. In the sparse pathway dispatch path (`pathway_top_k < 3`), `sparse_pathway_dispatch` groups tokens by selected pathway and executes only the needed pathways. However, this path also crashes due to an `index_add_` shape mismatch, indicating a second independent bug in the sparse dispatch implementation.

E. Component Audit

E.1 Transformer / Attention

LocalAttentionPathway uses standard multi-head scaled dot-product attention with separate Q/K/V projections, per-head LayerNorm on Q and K (a non-standard choice), window masking (asymmetric, equal forward and backward, combined with causal mask), and causal masking via a cached lower-triangular boolean mask. Head count is `num_attention_heads // 2` (half of model heads). No validation that `num_attention_heads` is even. Complexity is $O(T^2 * H)$ for the attention computation.

Position bias is supported but not generated by the model (always None).

E.2 SSM

SSMPathway implements a Mamba-style diagonal-A SSM with: state_dim configurable (default 16), input-dependent timestep via dt_proj with softplus activation, ZOH discretization with second-order Taylor expansion for numerical stability (ssm_scan.py), three scan methods: sequential ($T \leq 64$), chunked ($T > 64$, chunk=256), and parallel (Hillis-Steele). The _scan_method_str attribute that selects scan method is never set, always defaulting to "chunked". The center-padded Conv1d (padding=kernel_size//2) allows each position to see one future position, documented as a known causal violation. The SSM FLOP estimation overcounts by a factor of state_dim by treating diagonal A as if it were a full-rank matrix. A standalone S4DKernel implementation in ssm.py uses complex-valued A and has a more severe causal violation (padding=2 for kernel_size=3 allows seeing 2 future positions) but is unused dead code.

E.3 MoE

ShardedExpertFabric manages experts with disk-backed storage and LRU caching. In test_mode, a single dummy ExpertFFN (SwiGLU: $\text{down_proj}(\text{silu}(\text{gate_proj}(x)) * \text{up_proj}(x))$) processes all tokens, weighted by sum of routing weights. Expert count is configurable (default 192), top-k=2 by default, capacity factor 1.25. The load balance loss in zmoe.py uses an incorrect L2 formula against uniform distribution, NOT the standard Switch Transformer formula. A separate, correct Switch Transformer implementation exists in load_balance.py but the model-level loss is the incorrect L2 version (disabled when unify_load_balance=True). The LRU cache evicts experts without saving updated parameters to disk, meaning training updates can be silently lost on cache eviction. This is a critical correctness concern for any non-test_mode training scenario.

E.4 Routing

The AdaptiveRouter is the most complex component (~990 lines). It has a 3-layer MLP feature encoder (hidden dims scale with router_hidden_dim), and 5 routing heads: depth_router (sigmoid + STE binary per layer), width_router (Gumbel-softmax over width choices), path_router (Gumbel-softmax over 3 pathways), expert_router (top-k with capacity constraint), and complexity/uncertainty estimators. Temperature annealing is very slow: $\max(0.1, \text{temperature} * 0.99^{(\text{step}/1000)})$. A prior weight decays from 0.5 to 0.05 floor. Cost-aware routing adds biases based on a compute_budget scalar, but the pathway bias is inverted (see Section P). At eval, deterministic Gumbel noise (seed=1337+axis_id) is added to break routing collapse, generated on CPU then transferred. The _compute_balancing_loss method is deprecated (CV formula) but still callable. The _compute_efficiency_loss creates a leaf tensor with no gradient, providing no gradient signal.

E.5 Pathways

Three parallel pathways exist in each HASSBlock: Local (attention), LowRank (global attention in compressed space), and SSM (state space model). The pathway_gate that previously existed was removed in v0.5 (confirmed: no pathway_gate attribute on HASSBlock). Pathway selection is handled entirely by the router. When pathway_top_k ≥ 3 , all three pathways execute (no sparsity). When pathway_top_k < 3 , sparse_pathway_dispatch groups tokens by selected pathway. The sparse dispatch path is currently broken (index_add_shape mismatch), so genuine pathway sparsity cannot be tested. The pathway top-k mechanism exists but has not been validated to actually save wall-clock time, and the routing collapse at eval (without noise) means all tokens may select the same pathway, negating any sparsity benefit.

E.6 Depth

Adaptive depth is implemented via a binary depth_mask [B, T, max_depth] produced by the router with Gumbel-sigmoid STE. At inference, mask > 0.5 determines active layers. For fully active layers (all tokens), the normal block forward is used. For partially active layers, forward_with_depth gathers active tokens, processes them through the block, and scatters back. The minimum depth is enforced via torch.where. However, forward_with_depth has a critical bug: it creates a mini routing decision with expert_indices=torch.zeros(1, n, top_k), routing ALL extracted tokens to expert 0 regardless of their original expert assignment. This means depth sparsity and expert diversity are mutually exclusive in the current implementation. Whether depth masking genuinely reduces FLOPs is UNKNOWN because the forward pass is blocked.

E.7 Width

SlicedFFN implements genuine per-token width sparsity by slicing weight matrices: fc1.weight[:, :w] and fc2.weight[:, :w]. Width choices are derived from config.width_choices plus max_width. At inference, width_idx from the router determines per-token width. At training, hard argmax is used for the forward pass (genuine sparsity), with width_probs used only for auxiliary losses. The STE comment in the code admits that gradients do NOT flow through width_probs to the actual computation -- this is a hard selection with detached gradients, not a true STE. The inconsistency between _forward_single_width (applies ln_hidden at max_width) and _forward_single_width_flat (never applies ln_hidden) means the per-token width path and the fixed-width path produce different outputs at max_width. Whether width sparsity actually reduces wall-clock time is UNKNOWN because the forward pass is blocked.

F. Correctness Audit

F.1 Blocking Forward Pass Bug

[FAIL] LowRankGlobalPathway shape bug (CRITICAL)

In hass_block.py line 296, the causal mask is constructed as causal_mask[None, None, :, :] which has shape [1, 1, T, T]. The attention scores have shape [B, T, T] (3D). When masked_fill broadcasts the 4D mask against the 3D scores, PyTorch broadcasts to the larger shape [B, 1, T, T], introducing a spurious dimension. This 4D output propagates through softmax, matmul, and the from_low_rank projection, causing the LowRankGlobalPathway to return [B, 1, T, H] instead of [B, T, H]. The extra dimension then causes SlicedFFN._forward_per_token_width to crash with "ValueError: too many values to unpack (expected 3)" because it expects 3D input. Evidence: Directly reproduced by instrumenting the model: Local returns [1, 8, 128], LowRank returns [1, 1, 8, 128], SSM returns [1, 8, 128]. Fix: Change causal_mask[None, None, :, :] to causal_mask[None, :, :] (single unsqueeze). This bug is present in both the full-attention branch (line 296) and the chunked-attention branch (line 312).

F.2 Sparse Dispatch Bug

[FAIL] index_add_shape mismatch in sparse_pathway_dispatch

When pathway_top_k < 3, sparse_pathway_dispatch crashes with "index_add(): Number of indices (N) should equal source.size(dim): (1)". This is a second independent blocking bug preventing sparse pathway execution. The issue is in sparse_dispatch.py line 175 where index_add_ is called with mismatched index and source sizes. This prevents testing of genuine pathway sparsity. Evidence: 2 test failures in test_v05_fixes.py, plus direct reproduction with pathway_top_k=2 configuration. Status: UNKNOWN root cause without deeper investigation.

F.3 Causal Correctness

[UNKNOWN] Cannot test -- forward pass blocked

The intervention-based causal correctness test (modifying a future token and verifying earlier outputs remain unchanged) could not be performed because the model cannot complete a forward pass. All 6 causal correctness tests in the existing test suite fail (test_causal_leakage_fix.py). The LowRankGlobalPathway commit message claims to fix causal leakage, but the shape bug introduced in the same commit prevents verification. The SSM center-padding causal violation (+/-1 position) is documented but untested at HEAD. The standalone ssm.py SSMBlock has a more severe causal violation (padding=2 allows 2 future positions) but is dead code. Status: UNKNOWN for all components.

F.4 Gradient Flow

[UNKNOWN] Cannot test -- forward pass blocked

Backward pass and gradient flow could not be tested because the forward pass fails. From code analysis: (1) The depth masking uses correct STE (mask_hard - probs.detach() + probs) for gradient flow. (2) The SlicedFFN width routing admits gradients do NOT flow through width_probs to the actual computation (hard argmax, detached). (3) The efficiency_loss creates a detached leaf tensor providing no gradient. (4) The lm_loss field in ModelOutput is stored but NOT included in total_loss(), which may or may not be intentional. (5) Dead parameters (context_weights, injection_gate) receive initial gradients from weight initialization but are never used in computation, so their gradients are zero after the first backward pass. (6) The continuation.py imports a nonexistent xorzenTrainer alias, which would cause ImportError if that module were used.

F.5 Numerical Stability

Initial parameter inspection shows no NaN or Inf values. The ZOH discretization in ssm_scan.py uses second-order Taylor expansion with safe division for small values (eps=1e-4), which is numerically sound. The Gumbel noise generation clamps at 1e-10 to prevent log(0). However, the SSM FLOP estimation would overflow for large state_dim due to the overcounting bug. The parallel_scan uses O(T log T) memory due to 4 tensor clones per step. Without being able to run a forward pass, no runtime numerical stability assessment is possible. Status: UNKNOWN for runtime behavior.

G. Compute Audit

G.1 Parameters

Independent parameter count for a minimal test configuration (H=128, L=6, heads=8, vocab=1000, experts=8, top_k=2, CoT=6x32, low_rank=64, ssm_state=16, widths=[32,64,96,128]): Total: 2,846,555 parameters, 2,491,418 trainable. The model log reports the same counts, which match the independent sum. However, this includes dead parameters: context_weights in each LowRankGlobalPathway (6 blocks x low_rank_dim*4 parameters each), and injection_gate in CoT. The config.estimate_parameters() method only estimates a standard transformer and does NOT include MoE experts, router, CoT, merger, or HASS-specific parameters. Its output would be significantly below the actual count. The model log also reports "active params" based on top-k expert selection (e.g., "Top-2/8 experts active per token (~12.6% of trainable)") but this is a theoretical fraction based on parameter counting, not measured execution.

G.2 Active Parameters

[UNKNOWN] Cannot independently verify -- model cannot execute

The model reports an active parameter estimate via _estimate_active_params in model.py, but this method counts parameters in components based on routing decisions rather than measuring actual execution. The estimate assumes that if a pathway is selected, all its parameters are active, which is

correct for dense execution but does not account for matrix slicing in SlicedFFN (where active parameters vary by selected width). The `forward_with_depth` bug (all tokens routed to expert 0) means the active parameter accounting in the depth-sparse path is incorrect. An independent instrumented measurement is not possible because the forward pass is blocked.

G.3 FLOPs

[UNKNOWN] Theoretical estimates only, no runtime measurement possible

Theoretical FLOP estimation for the test config (B=1, T=32, dense mode, all pathways active): Per block: local attention ~262K FLOPs, low-rank projections ~1M FLOPs, low-rank attention ~131K FLOPs, SSM ~196K FLOPs, FFN (at max width 512) ~4.2M FLOPs. Total per block ~5.8M FLOPs. 6 blocks + LM head ~36M FLOPs. These are rough theoretical estimates for dense execution. The SSM FLOP estimate overcounts by a factor of `state_dim` (16x) due to treating diagonal A as full-rank. No runtime FLOP measurement is possible. The model's own `get_compute_stats` methods have known inaccuracies (hardcoded float32, SSM overcounting).

H. Memory Audit

[UNKNOWN] Only theoretical estimates possible

For the test configuration: parameter memory ~10.9 MB (float32). No runtime memory measurement is possible. The model does not use gradient checkpointing during this audit (it is enabled in the config but the forward pass fails before reaching that code). The `parallel_scan` in `ssm_scan.py` uses $O(T \log T)$ memory due to 4 tensor clones per step, which could be significant for long sequences. The MoE LRU cache holds expert parameters in memory; with 24 cached experts and SwiGLU FFN (`hidden_dim * 4 * 3` projections), memory usage depends on hidden size. No activation memory or peak memory measurements are possible without a working forward pass.

I. Training Audit

The `XORZENXTrainer` in `trainer.py` implements a standard training loop with: configurable optimizer (AdamW implied), learning rate scheduling, gradient accumulation, mixed precision (AMP), gradient clipping, periodic checkpointing, and epoch/step-based logging. The `TrainingState` dataclass tracks extensive metrics but has issues: `total_time` always returns 0 because `on_training_start()` is never called, and `steps_in_epoch/total_steps` are never updated through the proper callbacks (the trainer increments `global_step` directly). The epoch numbering on resume may double-count the checkpoint epoch. The `CurriculumTrainer` in `curriculum.py` implements book-by-book training with potential data leakage (no `ignore_index` for padding tokens in cross-entropy). The `continuation.py` has a broken import (`xorzenTrainer` does not exist). No training could be performed because the forward pass is blocked.

I.1 Loss Components

The total loss is: `lm_loss` (cross-entropy, shifted next-token prediction) + `routing_loss` (uncertainty regularizer) + router auxiliary losses (`load_balance_loss` via Switch formula, `router_z_loss`, `path_div_loss`, `width_div_loss`) + optionally `cot_consistency_loss`. The BUG-CRITICAL-1 fix (lines 617-626) correctly filters auxiliary losses to only include known scalar loss keys, preventing `features.mean()` from entering the loss. The `load_balance_loss` is correctly unified: when `unify_load_balance=True` (default), the model-level L2 loss is zeroed and only the Switch formula from the router is used. The `lm_loss` field in `ModelOutput` is stored but NOT included in `total_loss()`, which is either a bug or an intentional separation of LM loss from auxiliary losses. The CoT consistency loss is correctly disabled during pre-training (CoT is frozen, outputs zeros).

J. Inference Audit

[UNKNOWN] Cannot benchmark -- model cannot execute forward pass

No inference benchmarks are possible because the model cannot complete a forward pass at HEAD. All latency, throughput, and memory measurements are UNKNOWN. The generate() method supports greedy, sampling, and beam search decoding, but beam search has $O(\text{beams}^2)$ complexity with Python-level tensor concatenation (extremely slow). The per-token repetition penalty in `_apply_repetition_penalty` uses a Python loop over tokens (not GPU-vectorized). No KV-cache mechanism exists for autoregressive generation. The speed/ module provides C++ acceleration drop-ins (`fast_attention`, `fast_ssm`, `fast_router`, `fast_moe`) that require a compiled `xorzen_ext` C++ extension, which is not built in this environment. The `CachedRouter` uses MD5 hashing of full tensors on every inference step, which is slow. The `PreloadedExpertFabric` in `fast_moe.py` has broken code in `_fast_forward` (line 120-126) and references a non-existent `_prefetch_executor` in `__del__`.

K. Test Suite Results

The existing test suite contains 138 tests across 12 test files. 40 tests fail (29%), 98 pass (71%). The test execution took 8.40 seconds on the available hardware (CPU-only, 2 cores). The failure breakdown by file:

- `test_causal_leakage_fix.py`: 6/6 FAILED (all causal correctness tests)
- `test_cot_consistency_loss.py`: 6/6 FAILED (all CoT tests)
- `test_v05_fixes.py`: 6/6 FAILED (sparse dispatch, eval noise)
- `test_phase4_v04.py`: 7/7 FAILED (depth, width, training, cost-aware routing)
- `test_phase2_regression.py`: 4/4 FAILED (routing simplex, determinism)
- `test_phase2_sparsity.py`: 1/1 FAILED (STE gradient flow)
- `test_blocker_fixes.py`: 5/5 FAILED (routing loss, features in loss)
- `test_fixes.py`: 1/1 FAILED (SPPQ quantizer round-trip)
- `test_fix_p5_load_balance.py`: 2/2 FAILED (implied from blocker_fixes)
- `test_fix_sppq_schedule.py`, `test_fix_tokenizer_roundtrip.py`: PASS
- `test_phase1_correctness.py`: PASS

Many of the passing tests are likely testing trivial or standalone properties that do not exercise the full model forward path. The 6 passing tests in `test_phase1_correctness.py` are the most valuable as they verify basic model construction, but they do not perform a full forward pass. The failure of ALL causal correctness, ALL CoT, ALL sparse dispatch, and ALL phase-4 tests strongly suggests the model has been in a broken state since at least the commit that introduced the `LowRankGlobalPathway` shape bug.

L. Historical Claim Revalidation

Previous forensic audits reported several issues. Each is re-evaluated against the current HEAD commit. Note that many claims cannot be retested because the forward pass is blocked.

Historical Finding	Status	Evidence
Genuine pathway sparsity	UNKNOWN	Forward pass blocked; sparse dispatch also has independent crash
Genuine depth sparsity	UNKNOWN	Forward pass blocked; <code>forward_with_depth</code> has expert-routing bug
Incorrect active-parameter accounting	STILL TRUE	<code>forward_with_depth</code> routes all tokens to expert 0
Poor wall-clock performance at tiny scale	UNKNOWN	Cannot benchmark; no GPU; forward pass blocked
Causal masking in <code>LowRankGlobalPathway</code>	REGRESSED	New shape bug introduced in same commit as fix

Per-token routing	PARTIAL	Router produces per-token decisions but diversity untested
Weak input-dependent routing	UNKNOWN	Cannot test routing diversity without forward pass
Lack of real inference savings	STILL TRUE	Cannot measure; no working forward pass
Theoretical sparsity vs wall-clock gap	STILL TRUE	Still no evidence of wall-clock savings
Pathway gate removal	FIXED	No pathway_gate attribute on HASSBlock (verified)
Eval routing noise	PARTIAL	eval_routing_noise=0.15 exists but untested (forward blocked)
Load balance double-counting	FIXED	unify_load_balance=True correctly disables model-level L2 loss
CoT loss hardcoded to zero	FIXED	cot_consistency_loss computed conditionally (but CoT tests fail)

M. Scientific Claim Matrix

The following table evaluates every major scientific claim about XORZEN. Status classifications are strictly: PROVEN, PARTIALLY PROVEN, UNPROVEN, FALSE, or UNKNOWN. No "probably true" classifications are used.

Claim	Status	Evidence Basis
Per-token routing	PARTIALLY PROVEN	Router produces per-token decisions; diversity unmeasured
Input-dependent routing	UNKNOWN	Cannot test without forward pass
Genuine pathway sparsity	UNKNOWN	Sparse dispatch crashes; dense path always used
Genuine depth sparsity	UNKNOWN	Forward blocked; expert-routing bug in depth path
Genuine width sparsity	UNKNOWN	SlicedFFN code exists but untested at runtime
Active parameter reduction	UNPROVEN	Accounting has known bugs; no runtime measurement
FLOP reduction	UNPROVEN	FLOP counters have known inaccuracies; no runtime data
Inference compute reduction	UNPROVEN	Cannot benchmark without working forward pass
Wall-clock speedup	UNPROVEN	No benchmarks possible; no GPU; forward broken
Memory reduction	UNKNOWN	No runtime memory measurement possible
Training efficiency	UNKNOWN	No training possible; forward pass broken
Quality improvement	UNKNOWN	No training or evaluation possible
Quality/FLOP improvement	UNKNOWN	Cannot measure either quantity
Quality/parameter improvement	UNKNOWN	Cannot measure either quantity

Summary: 0 claims are PROVEN. 1 claim is PARTIALLY PROVEN (per-token routing, because the router code exists but diversity is untested). 3 claims are UNPROVEN (active params, FLOPs, wall-clock -- the accounting mechanisms exist but have known bugs and no runtime validation). 11 claims are UNKNOWN (cannot be tested due to the blocking forward pass bug and hardware limitations). 0 claims are FALSE (none could be disproven because they could not be tested). The XORZEN codebase makes many implicit claims through its naming ("genuine sparsity", "adaptive depth", "fast"); none of these performance claims are currently supported by experimental evidence.

N. Architectural Strengths

Despite the current broken state, the architecture contains several genuinely interesting and potentially valuable research ideas that are worth preserving once the implementation is stabilized.

1. Multi-axis adaptive routing. The idea of a single router controlling depth, width, pathway, and expert selection simultaneously is genuinely interesting. If the routing decisions are truly input-dependent and diverse, this could enable meaningful compute savings. The router architecture itself (3-layer encoder + 5 heads) is reasonable and could be effective with proper training. The cost-aware budget mechanism is a good conceptual framework for controlling compute-quality tradeoffs at inference time.
2. HASS parallel pathways. The combination of local attention (efficient for nearby dependencies), low-rank global attention (efficient for long-range dependencies in compressed space), and SSM (efficient for sequential patterns) is a well-motivated architecture. Each pathway addresses a different type of dependency, and adaptive selection between them could be more efficient than using all three for every token.
3. SlicedFFN matrix slicing. The approach of slicing weight matrices to achieve genuine per-token width sparsity (rather than computing full-width activations and masking) is technically sound and would produce real FLOP savings if the routing is diverse and the overhead is manageable. This is an implementation novelty worth preserving.

4. Internal Latent CoT. The idea of maintaining an internal reasoning state (6 components: intention, decomposition, confidence, contradiction, direction, summary) that evolves across layers is a novel approach to implicit reasoning. The component-specific activations (sigmoid for confidence, tanh for intention, normalize for direction) are well-chosen. Freezing CoT during pre-training and enabling it for fine-tuning is a reasonable training strategy.

5. Extensive self-auditing infrastructure. The repository contains 29+ historical audit reports, multiple audit scripts, and a test suite. The commit messages show awareness of specific bugs and a pattern of iterative fixing. The BUG-CRITICAL-1 fix (filtering auxiliary loss keys) shows careful attention to gradient flow. The Switch Transformer load balance formula is correctly implemented in `load_balance.py`.

O. Architectural Weaknesses

CRITICAL

- **[FAIL]** LowRankGlobalPathway shape bug: `causal_mask[None, None, :, :]` adds spurious 4th dimension, blocking all forward passes (`hass_block.py:296, 312`)
- **[FAIL]** Sparse dispatch `index_add_` crash: second independent blocking bug preventing pathway sparsity testing (`sparse_dispatch.py:175`)
- **[FAIL]** `forward_with_depth` routes all tokens to expert 0, losing per-token expert routing in depth-sparse path (`hass_block.py:1168`)
- **[FAIL]** LRU expert cache silently discards training updates on eviction -- no auto-save to disk (`zmoe.py:201`)
- **[FAIL]** 6 instances of `torch.load()` without `weights_only=True` -- arbitrary code execution vulnerability
- **[FAIL]** 40/138 tests (29%) fail, including ALL causal, ALL CoT, ALL sparse dispatch, and ALL phase-4 tests

HIGH

- **[FAIL]** Cost-aware routing pathway bias is INVERTED: biases toward expensive local attention under low budget instead of cheap SSM (`routing.py:572-576`)
- **[FAIL]** SlicedFFN `ln_hidden` inconsistency: applied in `_forward_single_width` at `max_width`, never in `_forward_single_width_flat` (per-token path)
- **[FAIL]** Dead parameter `context_weights` in LowRankGlobalPathway: registered `nn.Parameter`, initialized, never used in forward, inflates parameter counts
- **[FAIL]** Dead parameter `injection_gate` in InternalLatentCoT: defined, initialized, never called in forward
- **[FAIL]** `continuation.py` imports nonexistent `xorzenTrainer` -- broken import path
- **[FAIL]** `__init__.py` exports `check_installation` which is never defined -- `AttributeError` on import
- **[FAIL]** `ModelOutput.total_loss()` does not include `lm_loss` -- either bug or unclear design intent
- **[FAIL]** `_active_params_history` list grows unboundedly in `model.py` -- memory leak in long training

MEDIUM

- `Config.estimate_parameters()` and `estimate_flops_per_token()` only estimate standard transformer, not the actual XORZEN architecture
- `HASSBlockManager` is not an `nn.Module` -- blocks not registered as submodules
- `SSM_scan_method_str` never set, always defaults to "chunked"

- SSM FLOP estimation overcounts by factor of state_dim
- Gumbel noise always generated on CPU then transferred to device
- Per-token repetition penalty uses Python loop (not vectorized)
- Beam search $O(\text{beams}^2)$ with Python-level tensor concatenation
- CurriculumTrainer cross-entropy has no ignore_index for padding
- TrainingState.total_time always returns 0 (on_training_start never called)
- ssm.py SSMBlock has future-leaking Conv1d (padding=2 for kernel=3) -- dead code but still present
- Troll comment in cot_vector.py line 130
- zero_1_3B class name says 1.3B but PARAM_COUNT says 1B

P. Technical Debt

The repository has accumulated significant technical debt across multiple dimensions. The 29+ historical audit reports and 40+ experiment/analysis scripts indicate a pattern of rapid iteration without sustained testing discipline. Key debt items include: (1) The entire ssm.py file is dead code (S4DKernel and SSMBlock not used by the main model). (2) The IGRIS model (xorzen/models/igris/) is experimental and partially implemented, with dead parameters and missing features. (3) The speed/ module provides C++ acceleration that requires compilation (setup_speed.py) but the xorzen_ext extension is not built. (4) The train_superintelligence.py file is aspirational/marketing code that imports non-existent modules. (5) Multiple conflicting train() and continue_train() APIs exist across __init__.py and continuation.py. (6) The SPPQ quantization system (~2000 lines) is experimental and not integrated into the training pipeline. (7) The data/ module has extensive format support but is not used by the main training path. (8) The __init__.py imports all model variants at package import time, causing heavy import cost for what should be a simple import.

Q. Potential Novelty

Each mechanism is classified by its apparent novelty level. This assessment is based on code analysis, not a comprehensive literature search.

- Multi-axis adaptive router (depth + width + pathway + expert): Potentially novel. Combining all four routing axes into a single router with a shared feature encoder is not a standard architecture. The closest precedent is UL2/Mixture-of-Experts with conditional computation, but the width and pathway axes are less common. Requires literature investigation.
- SlicedFFN per-token width via matrix slicing: Implementation novelty. Using weight matrix slicing (`fc1.weight[:w, :]`) for genuine per-token width sparsity is a clean implementation approach. The concept of variable-width FFN is established (SlimPajama, etc.), but this specific slicing mechanism may be novel.
- Internal Latent CoT with 6 semantic components: Potentially novel. Having an explicit multi-component internal reasoning state that evolves across layers is not standard. Closest precedents are Pause Tokens and internal monologue, but the 6-component structure with component-specific activations is unusual.
- HASS parallel pathways (Local + LowRank + SSM): Combination of established techniques. Each pathway is individually well-established. The combination into a single block with adaptive selection is a design choice, not a novel mechanism.
- Cost-aware routing budget: Established concept (conditional computation, universal transformers). The specific implementation with a scalar budget bias is an engineering choice.
- Sharded MoE with disk-backed LRU cache: Combination of established techniques. Disk-offloading experts is explored in the literature (DeepSpeed-MoE, etc.).

R. SOTA Gap

XORZEN cannot be evaluated against SOTA because it cannot execute a forward pass. The following gaps exist and would need to be addressed before any SOTA comparison is meaningful. (1) Capability gap: No training or evaluation has been done at any scale at HEAD. The largest experiment in the repository is 10M tokens on TinyStories, which is insufficient for any capability claim. (2) Parameter-efficiency gap: Unknown -- no training baseline exists. Would need comparison against standard transformers, LLaMA-style architectures, and MoE models (Mixtral, DeepSeek) at comparable parameter counts. (3) Compute-efficiency gap: Unknown -- no wall-clock measurements exist. Would need comparison against FlashAttention-2, Mamba, and other efficient architectures. (4) Inference-efficiency gap: Unknown -- no KV-cache, no speculative decoding, no quantization in the inference path. (5) Memory-efficiency gap: Unknown -- no peak memory measurements. Would need comparison against parameter-sharing and quantized baselines. (6) Training-efficiency gap: Unknown -- no training runs at HEAD. The current experiments are too small to establish any gap.

S. Recommended Research Priorities

Priority 1: Fix the blocking forward pass bug

Problem: LowRankGlobalPathway shape bug blocks all forward passes and testing. Hypothesis: Changing `causal_mask[None, None, :, :]` to `causal_mask[None, :, :]` in lines 296 and 312 will fix the forward pass. Why it matters: Without a working forward pass, no other research can proceed. Required experiment: Fix the mask, run full forward pass, verify output shapes. Expected evidence: Clean forward pass with correct output shapes [B, T, V]. Success criterion: Forward pass completes, existing tests begin passing. Failure criterion: Different shape error or NaN output. Estimated compute cost: Negligible (code fix + unit test).

Priority 2: Achieve 100% test pass rate

Problem: 40/138 tests fail. Many test architectural properties that must be verified before any benchmarking. Hypothesis: Most failures are downstream of the shape bug and will resolve after the fix. Why it matters: Tests are the only automated correctness verification. Required experiment: Fix shape bug, re-run full test suite, investigate remaining failures. Expected evidence: Test pass rate increases significantly. Success criterion: 100% pass rate or documented reasons for remaining failures.

Priority 3: Verify causal correctness

Problem: Causal correctness is a first-class requirement but entirely untested. Required experiment: Intervention test: modify future token, verify earlier outputs unchanged, for each component independently. Success criterion: All causal tests pass with tolerance $1e-5$. Failure criterion: Any future-to-past leakage detected.

T. NOW / NEXT / LATER / REJECTED

NOW: Fix the LowRankGlobalPathway shape bug (causal_mask broadcasting). This is the single highest-priority action.

NEXT: Fix the sparse dispatch `index_add_` crash. Fix the `forward_with_depth` expert-routing bug. Remove dead parameters. Fix SlicedFFN `In_hidden` inconsistency. Fix cost-aware routing bias inversion.

LATER: Add `torch.load weights_only=True`. Fix `continuation.py` import. Remove dead `ssm.py`. Add causal correctness intervention tests. Add routing diversity tests. Benchmark on GPU. Compare against dense transformer and standard MoE baselines.

REJECTED (do NOT work on yet): New architectural mechanisms. Performance optimization. Scaling experiments. SOTA comparisons. New features. The model must first work correctly before any of these are meaningful.

U. Final Verdict

Current XORZEN maturity: Experimental prototype.

Most important verified achievement: The multi-axis adaptive routing architecture (depth, width, pathway, expert) is a genuinely interesting and potentially novel research direction. The codebase demonstrates sustained development effort with extensive self-auditing. The SlicedFFN matrix-slicing approach to width sparsity is technically sound.

Most serious verified flaw: The `LowRankGlobalPathway` shape bug (`hass_block.py:296`) makes the model completely non-functional. This single bug prevents any forward pass, training, evaluation, or benchmarking. Combined with the sparse dispatch crash, it means the model has been in a broken state at HEAD, and all historical performance claims are based on code that no longer works.

Most promising research hypothesis: That the multi-axis adaptive router, once stabilized, can achieve meaningful compute savings by routing easy tokens through cheap pathways (SSM, narrow width, shallow depth) while reserving expensive pathways (local attention, wide width, deep processing) for complex tokens. This hypothesis is currently neither proven nor disproven.

Biggest measurement gap: No wall-clock performance data exists for any configuration. All compute savings claims are theoretical. The gap between theoretical sparsity and actual wall-clock savings is the most important unknown.

First thing that should be fixed: The `causal_mask` broadcasting bug in `hass_block.py` lines 296 and 312. Change `causal_mask[None, None, :, :]` to `causal_mask[None, :, :]`.

First experiment after the audit: Run a complete forward pass on a clean configuration and verify: (1) correct output shapes, (2) no NaN/Inf, (3) causal correctness via intervention test, (4) gradient flow to all trainable parameters, (5) routing diversity across tokens and inputs.

What should NOT be worked on yet: New features, architectural changes, performance optimization, scaling experiments, SOTA comparisons, or any benchmarking. The model must first achieve a clean forward/backward pass, pass all tests, and demonstrate causal correctness before any further development.

Repository: github.com/akikfaraji/DevNet | Commit: `4b2bc57c806059b49b3ad375293147b62e133575` | Environment: Debian 13, Python 3.12.14, PyTorch 2.13.0+cpu, 2x Intel Xeon cores, 4.1 GiB RAM, no GPU | Audit date: 2026-09-01